Лабораторная работа 5

Графовые нейронные сети

Насколько результат определяется структурой графа

Курс «Нейронные сети»

Редакция 4 от 8 октября 2026 г.

1 Введение

Многие данные — это объекты и связи: статьи ссылаются друг на друга. Такие данные описывают графом: объекты — вершины, связи — рёбра. Графовая нейронная сеть (graph neural network, GNN) считает представление вершины по её признакам и по соседям. Каждый слой передаёт им «сообщения», поэтому его называют передачей сообщений (message passing).

Пример — набор данных Cora: 2708 научных статей и 5278 ссылок между ними. Каждая статья относится к одной из семи тем, но тема известна только у 140 статей. Темы остальных нужно предсказать. 81 % ссылок соединяют статьи одной темы, а у случайных пар статей тема совпадала бы лишь в 18 % случаев. Такую склонность связанных вершин принадлежать одному классу называют гомофилией (homophily).

Высокая точность ещё не доказывает, что помог граф. Выигрыш могли дать признаки, усреднение или число соседей. Поэтому меняют только рёбра: их удаляют или перемешивают, сохраняя степени. Так получают нулевую модель (null model). Если слоёв слишком много, представления вершин почти совпадают. Это называют чрезмерным сглаживанием (oversmoothing).

Все эксперименты выполняются в проекте из ЛР0. У каждого запуска есть `EXP_ID` и `RUN_ID`. Числа в отчёте сопровождаются ссылками на `RUN_ID`.

2 Цель работы

Цель работы — экспериментально установить, какую информацию графовая сеть получает из структуры графа сверх признаков вершин. Вторая цель — найти глубину, после которой передача сообщений перестаёт помогать. Итог работы — **карта зависимости качества от структуры графа**, а не таблица «GCN против GAT».

Когда структура графа содержит полезную информацию, которую нельзя получить только из признаков вершин?

3 Что нужно знать заранее

Если какой-то пункт незнаком, восполните пробел по указанному источнику.

- Матрицы, собственные значения и собственные векторы — [2, разд. 2.2, 2.7].
- Графы, матрица смежности (adjacency matrix), задачи для вершин и графов — обзор [4].
- Перцептрон, функции активации, оптимизатор Adam — [1, разд. 3.3, 4.5], [2, разд. 6.3, 8.5].
- Прореживание (dropout) и штраф на веса (weight decay) — [1, разд. 4.1], [2, разд. 7.12].
- Классификация вершин, когда метки известны лишь у части из них, — [4, 6].
- Механизм внимания (attention), если профиль назначил GAT, — [1, разд. 8.1].
- Библиотека PyTorch Geometric (PyG): объект `Data`, тензор `edge_index`, маски вершин, слой `GCNConv` — руководство [3].
- Проект из ЛР0; монитор градиентов из ЛР1 и показатели ρ_T и GI (формулы (1.14) и (1.16) ЛР1).

4 Результаты обучения

После выполнения работы вы сможете:

1. записывать GCN, GraphSAGE и GAT через передачу сообщений;
2. сравнивать графовую сеть с перцептроном без графа и с нулевой моделью;
3. измерять гомофилию графа и сравнивать её с уровнем случайности;
4. повреждать граф и проверять реализацию по предсказанным величинам;
5. отличать чрезмерное сглаживание от переобучения и плохой обучаемости;
6. соблюдать трансдуктивный (transductive) протокол: не использовать метки вершин `test` ни для какого выбора;
7. делать вывод о роли графа по трём seeds и парным разностям.

5 Модели и обозначения

5.1 Граф, признаки и метки

Граф $G = (V, E)$ — это множество вершин V, $N = |V|$, и множество рёбер E. Матрица смежности A размера $N \times N$ описывает рёбра: $A_{ij} = 1$, если вершины i и j соединены, иначе $A_{ij} = 0$. Графы в работе неориентированные: матрица A симметрична. Число соседей вершины называют её *степенью*:

$$d_i = \sum_{j=1}^{N} A_{ij}, \qquad \mathcal{N}(i) = \{\, j : A_{ij} = 1 \,\}, \qquad \sum_{i=1}^{N} d_i = 2|E|, \tag{5.1}$$

где $\mathcal{N}(i)$ — множество соседей вершины i, $|E|$ — число неориентированных рёбер. У вершины есть вектор признаков $x_i \in \mathbb{R}^F$ (строка матрицы X) и метка $y_i \in \{1, \dots, C\}$; N_c — число вершин класса c.

В PyG рёбра лежат в `edge_index` размера $2 \times 2|E|$: каждая пара записана как (i, j) и как (j, i). Петли (self-loops) в данных не хранят, их добавляет модель. Вершина с $d_i = 0$ изолирована: сеть видит только её признаки.

Подробнее: [2, разд. 2.2]; [4, 3].

5.2 Передача сообщений

Простыми словами, на каждом слое вершина собирает сообщения соседей, объединяет их и обновляет своё представление [5]:

$$m_i^{(l)} = \mathrm{AGG}\big(\{\,\psi(h_i^{(l)}, h_j^{(l)}) : j \in \mathcal{N}(i)\,\}\big), \qquad h_i^{(l+1)} = \phi\big(h_i^{(l)}, m_i^{(l)}\big). \tag{5.2}$$

Здесь $h_i^{(l)}$ — представление вершины i после слоя l, $h_i^{(0)} = x_i$. Матрица из этих строк — $H^{(l)}$. Функция ψ строит сообщение, AGG не зависит от порядка соседей. Функция ϕ обновляет представление. Перцептрон (MLP) сообщений не использует.

После L слоёв на вершину влияют только вершины на расстоянии не больше L:

$$h_i^{(L)} = F_L\big(\{\,x_j : j \in B_L(i)\,\},\ G[B_L(i)]\big), \qquad B_L(i) = \{\,j : \mathrm{dist}(i,j) \le L\,\}, \tag{5.3}$$

где $\mathrm{dist}(i,j)$ — число рёбер на кратчайшем пути между вершинами, $G[B_L(i)]$ — рёбра внутри $B_L(i)$, F_L — функция, которую вычисляет сеть. Множество $B_L(i)$ называют рецептивным полем (receptive field). Достижимость не гарантирует, что сведения сохранятся: их может стереть усреднение (раздел 5.6). Это проверяет эксперимент E3.

5.3 Три графовые модели

GCN [6] усредняет вершину с её соседями и применяет общий линейный слой:

$$\tilde{A} = A + I, \qquad \tilde{D} = \mathrm{diag}(d_1 + 1, \ldots, d_N + 1), \qquad \hat{A} = \tilde{D}^{-1/2}\tilde{A}\,\tilde{D}^{-1/2},$$
$$H^{(l+1)} = \sigma\big(\hat{A} H^{(l)} W^{(l)}\big), \tag{5.4}$$

где I — единичная матрица (петля у каждой вершины), $\tilde{D}$ — степени с учётом петель, $W^{(l)}$ — обучаемые веса слоя, σ — ReLU. На последнем слое нелинейности нет: он выдаёт оценки классов. Вес соседа j равен $\hat{A}_{ij} = 1/\sqrt{(d_i+1)(d_j+1)}$, вес самой вершины — $1/(d_i+1)$. Если рёбер нет, то $\hat{A} = I$, и слой GCN совпадает со слоем перцептрона $H^{(l+1)} = \sigma(H^{(l)}W^{(l)})$.

GraphSAGE [7] обрабатывает вершину и среднее её соседей раздельно:

$$m_i^{(l)} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} h_j^{(l)}, \qquad h_i^{(l+1)} = \sigma\big(W^{(l)}\,[\,h_i^{(l)};\, m_i^{(l)}\,]\big), \tag{5.5}$$

где $[\,\cdot\,;\,\cdot\,]$ — объединение двух векторов в один (конкатенация); у изолированной вершины $m_i^{(l)} = 0$. Вершина и соседи умножаются на разные части $W^{(l)}$, поэтому модель может ослабить вклад бесполезных соседей. В GCN матрица у них общая.

GAT [8] взвешивает соседей обучаемыми весами внимания. Для одной головы внимания:

$$e_{ij} = \mathrm{LeakyReLU}\big(a^\top [\,W h_i;\, W h_j\,]\big), \qquad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)\cup\{i\}} \exp(e_{ik})},$$
$$h_i' = \sigma\Big(\sum_{j \in \mathcal{N}(i)\cup\{i\}} \alpha_{ij}\, W h_j\Big), \tag{5.6}$$

где W — матрица весов, a — обучаемый вектор внимания, α_{ij} — вес внимания соседа j. Веса положительны и в сумме по соседям и самой вершине дают 1. Выходы нескольких голов в скрытых слоях объединяются, в последнем слое — усредняются.

Подробнее: [5, 6, 7, 8]; слои PyG — [3].

5.4 Гомофилия

Простая мера гомофилии — доля рёбер внутри классов [9]:

$$h_E = \frac{|\{\,(i,j) \in E : y_i = y_j\,\}|}{|E|}, \tag{5.7}$$

где в числителе — число рёбер, оба конца которых имеют одну метку. Малое h_E означает гетерофилию (heterophily): рёбра чаще соединяют разные классы.

Значение h_E сравнивают с уровнем случайности:

$$h_0 = \sum_{c=1}^{C} \Big(\frac{N_c}{N}\Big)^2, \qquad h_{\deg} = \sum_{c=1}^{C} \Big(\frac{\mathrm{vol}_c}{2|E|}\Big)^2, \qquad \mathrm{vol}_c = \sum_{i:\, y_i = c} d_i, \tag{5.8}$$

где vol_c — сумма степеней вершин класса c. Величина h_0 приближённо равна h_E графа, рёбра которого соединяют случайные пары вершин. Величина $h_{\deg}$ — то же ожидание для графа с прежними степенями. Конец ребра попадает в вершину i с вероятностью, пропорциональной d_i. При неравных классах $h_0 > 1/C$, и высокое h_E отчасти случайно. Для Cora $h_E = 0{,}81$, $h_0 = 0{,}18$, $h_{\deg} = 0{,}17$. Эти величины считают по всем меткам и ничего по ним не выбирают.

Подробнее: [9]; [4].

5.5 Возмущения графа и нулевые модели

Удаление рёбер. Каждое неориентированное ребро удаляется независимо от других с вероятностью p; обе записи ребра, (i,j) и (j,i), удаляются вместе:

$$E_p = \{\, e \in E : \xi_e = 1 \,\}, \qquad \xi_e \sim \mathrm{Bernoulli}(1-p), \tag{5.9}$$

где ξ_e — независимые случайные величины: 1 — ребро сохраняется, 0 — удаляется. Вершина становится изолированной, если удалены все её d_i рёбер; вероятность этого p^{d_i}. Отсюда ожидаемое число изолированных вершин:

$$\mathbb{E}\big[N_{\mathrm{iso}}(p)\big] = \sum_{i=1}^{N} p^{d_i}, \tag{5.10}$$

где d_i — степень в исходном графе; для вершин, изолированных с самого начала, $p^0 = 1$. Для Cora при $p = 0{,}5$ формула даёт около 494 изолированных вершин, то есть 18 % графа. Случайное удаление в среднем не меняет h_E: рёбра внутри классов и между ними удаляются одинаково часто.

Нулевая модель с сохранением степеней. Рёбра перемешивают: число соседей каждой вершины прежнее, сами соседи случайные. Двойная перестановка (double edge swap) [10] заменяет два ребра (a,b) и (c,d) парой

$$\{(a,b),\ (c,d)\} \ \longrightarrow\ \{(a,d),\ (c,b)\}, \tag{5.11}$$

если новые рёбра не петли и ещё не входят в граф. Степени всех четырёх вершин не меняются. Выполняют $Q|E|$ попыток, $Q = 10$, и записывают долю успешных. Ориентацию второго ребра выбирают случайно. После перемешивания h_E близка к $h_{\deg}$ (5.8). Более слабая нулевая модель — граф Эрдёша–Реньи из $|E|$ случайных пар. Степени он не сохраняет, а его h_E близка к h_0.

Прирост от графа. Первое сравнение — с перцептроном на тех же признаках и масках:

$$G_{\text{top}} = M_{\text{GNN}}(G) - M_{\text{MLP}}, \tag{5.12}$$

где M — доля правильных ответов (accuracy) на вершинах `test`. Положительный G_{top} ещё не доказывает пользу конкретной структуры: выигрыш могли дать усреднение или степени. Проверка — сравнение с нулевой моделью при общих seeds $s = 1, 2, 3$, как в формуле (1.20) ЛР1:

$$\begin{gathered} \Delta_s = M_{\text{GNN}}(G, s) - M_{\text{GNN}}\big(G^{(s)}_{\text{null}}, s\big), \\ \bar{\Delta} = \frac{1}{3}\sum_{s=1}^{3} \Delta_s, \qquad s_\Delta = \sqrt{\frac{1}{2}\sum_{s=1}^{3}(\Delta_s - \bar{\Delta})^2}, \end{gathered} \tag{5.13}$$

где seed s задаёт инициализацию, прореживание и нулевой граф $G^{(s)}_{\text{null}}$, $\bar{\Delta}$ — прирост от конкретной структуры рёбер, s_Δ — стандартное отклонение разностей.

Подробнее: [10]; зависимость результатов от разбиения — [11].

5.6 Чрезмерное сглаживание

Каждый слой GCN усредняет вершину с её соседями. После многих слоёв представления вершин одной компоненты связности становятся почти одинаковыми [12]. Без весов и нелинейностей L слоёв GCN — это умножение на $\hat{A}^L$. Собственные значения $\hat{A}$ лежат в промежутке $(-1, 1]$. Значению 1 отвечает собственный вектор с элементами $\sqrt{d_i + 1}$, по одному на каждую компоненту связности, а значение -1 исключают петли. Поэтому для связного графа

$$\begin{gathered} \lim_{L\to\infty} \hat{A}^L = \frac{\tilde{D}^{1/2}\mathbf{1}\mathbf{1}^\top \tilde{D}^{1/2}}{\sum_j (d_j + 1)}, \\ \lim_{L\to\infty} \big(\hat{A}^L X\big)_i = \sqrt{d_i + 1}\, v, \qquad v = \frac{\sum_j \sqrt{d_j + 1}\, x_j}{\sum_j (d_j + 1)}, \end{gathered} \tag{5.14}$$

где $\mathbf{1}$ — вектор из единиц, а v — общий вектор одной компоненты. Строки отличаются только множителем $\sqrt{d_i + 1}$. Если компонент несколько, суммы в v берут внутри компоненты. Отклонение убывает как λ^L, где λ — наибольший модуль остальных собственных значений $\hat{A}$. С весами и ReLU то же доказано, если наибольшее сингулярное число $W^{(l)}$, умноженное на λ, меньше единицы [13]. Обученные веса могут нарушать это условие, поэтому сглаживание измеряют.

Для представлений слоя $H = H^{(l)}$ используются три измерения. **Средняя косинусная близость** показывает, насколько одинаково направлены векторы разных вершин:

$$S(H) = \frac{1}{N(N-1)} \sum_{i \neq j} \frac{h_i^\top h_j}{\|h_i\|\,\|h_j\|}, \tag{5.15}$$

где $\|\cdot\|$ — длина вектора. В пределе (5.14) получается $S = 1$. На больших графах S считают по 20 000 случайных пар. После ReLU координаты неотрицательны, поэтому S сравнивают между слоями, а не с нулём.

Нормированная энергия Дирихле (Dirichlet energy) графа с петлями [14] показывает, насколько различаются представления соседей по сравнению с величиной

самих представлений:

$$\hat{\mathcal{E}}(H) = \frac{\sum_{(i,j)\in E} \left\| h_i/\sqrt{d_i+1} - h_j/\sqrt{d_j+1} \right\|^2}{\sum_{i=1}^{N} \|h_i\|^2} = \frac{\operatorname{tr}\bigl(H^\top (I - \hat{A}) H\bigr)}{\|H\|_F^2}, \tag{5.16}$$

где сумма берётся по неориентированным рёбрам, tr — след матрицы, $\|H\|_F$ — норма Фробениуса. Значения $\hat{\mathcal{E}}$ лежат в $[0, 2)$. Ноль достигается в пределе (5.14): $h_i = \sqrt{d_i + 1}\, v$ в каждой компоненте. Постоянные $h_i = v$ дают ноль, только если у концов каждого ребра равные степени. Умножение H на число не меняет $\hat{\mathcal{E}}$, поэтому её сравнивают между слоями разной ширины.

Линейный зонд (linear probe) — логистическая регрессия на представлениях слоя. Её обучают на `train` и проверяют на `val`. Сглаживание признают, когда падают и $\hat{\mathcal{E}}$, и точность зонда.

Подробнее: [12, 13, 14]; [4].

5.7 Сводка обозначений

Обозначение	Смысл
$G = (V, E)$, N, $\lvert E\rvert$, A	граф, число вершин и рёбер, смежность
d_i, $\mathcal{N}(i)$, X, x_i, y_i	степень, соседи, признаки, метка
C, N_c, vol_c	число классов, размер и сумма степеней класса
$h_i^{(l)}$, $H^{(l)}$, $W^{(l)}$, L, α_{ij}	представления, веса, глубина, вес внимания
$\tilde{A}$, $\tilde{D}$, $\hat{A}$, λ	граф с петлями, матрица GCN, скорость сходимости
h_E, h_0, $h_{\deg}$, p, N_{iso}	гомофилия, случайность, удаление
M, G_{top}, Δ_s, S, $\hat{\mathcal{E}}$	качество, прирост, разности, сглаживание

6 Данные и оборудование

Основные данные — один из трёх графов цитирования [15], по профилю. Вершина — научная статья, ребро — ссылка между статьями без учёта направления. Признаки вершины показывают, встречается ли в тексте статьи каждое слово словаря; в PubMed вместо 0 и 1 стоят веса слов TF-IDF. Метка — тема статьи.

Набор	Вершин	Рёбер	Признаков	Классов	`train` / `val` / `test`
Cora	2708	5278	1433	7	140 / 500 / 1000
CiteSeer	3327	4552	3703	6	120 / 500 / 1000
PubMed	19 717	44 324	500	3	60 / 500 / 1000

В таблице — неориентированные рёбра без петель и повторов. У Кипфа и Веллинга для Cora указано 5429 рёбер, файл Planetoid содержит 5278. Берите число из загруженного графа. Компонент связности: у Cora 78, у CiteSeer 438, у PubMed одна. Изолированных вершин нет у Cora и PubMed, у CiteSeer их 48.

Маска (mask) — список номеров вершин одной роли. Метки вершин `train`, по 20 на класс, входят в функцию потерь. По вершинам `val` выбирают гиперпараметры, эпоху и глубину. Вершины `test` служат только для итоговых измерений. Метки остальных вершин не используются. Маски — стандартные маски Planetoid [16], скопированные в

файл разбиения курса («Протокол данных курса»). Роль `cal` для графов пуста; как и во всех работах до ЛР8, её не читают.

Режим работы — трансдуктивный. Модель видит признаки и рёбра всех вершин, включая `test`, но функция потерь считается только по `train`. Метки `test` не участвуют ни в каком выборе: гиперпараметров, эпохи, глубины, p или Q. В индуктивном (inductive) режиме вершины `test` и их рёбра появляются только при оценке. Маски не меняют: результат зависит от разбиения и seed [11]. Сравнения ведут при общих seeds.

Оборудование. Видеокарта не нужна. Двухслойная GCN за 200 эпох на процессоре занимает примерно от 10 до 50 с. Нулевой граф при $Q = 10$ строится за секунды.

7 Индивидуальный профиль

Профиль ЛР5 задаёт три параметра. Добавьте их в генератор профиля из ЛР0. Используется функция `lab_pick` из ЛР1.

```
profile["lab5_dataset"] = lab_pick(sid, 5, 0, 2, ["cora", "citeseer", "pubmed"])
profile["lab5_second_gnn"] = lab_pick(sid, 5, 2, 4, ["sage", "gat"])
profile["lab5_depths"] = lab_pick(sid, 5, 4, 6, [[2, 4, 8, 16], [2, 3, 6, 12, 24]])
```

Здесь `sid` — номер зачётки. Профиль задаёт набор, вторую модель и глубины Е3. Основная модель — GCN. Смена профиля не меняет код.

8 Задания

8.1 Задание 1. Графовые модели: MLP, GCN и GraphSAGE или GAT

Что сделать.

1. Загрузите набор из профиля классом `Planetoid` [3]. Маски возьмите из файла разбиения курса и сверьте с масками `Planetoid`. Подготовку графа выполняйте в одном месте: симметризация, удаление петель и повторов, подсчёт изолированных вершин.

2. Реализуйте три модели с общим интерфейсом:

   ```
   logits = model(x, edge_index)
   hidden = model.forward_features(x, edge_index)  # list: one tensor per layer
   ```

   Перцептрон принимает `edge_index`, но не использует его. GCN строится из слоёв `GCNConv` (5.4), вторая модель — из `SAGEConv` (5.5) или `GATConv` (5.6). Петли добавляет сам слой (параметр `add_self_loops`), а не данные.

3. Общие настройки: глубина 2 (в Е3 — по профилю), ширина 64, прореживание 0,5, оптимизатор Adam, до 200 эпох. Выбирается эпоха с наибольшей точностью на `val`. У GAT в скрытом слое 8 голов по 8 признаков [8]. У перцептрона та же глубина и ширина.

4. Скорость обучения и штраф на веса подберите для каждой модели по одной сетке $\{0{,}01;\ 0{,}005\} \times \{5 \cdot 10^{-4};\ 5 \cdot 10^{-3}\}$ по точности на `val`. Функция потерь считается только по вершинам `train`.

Что должно получиться. Любая из трёх моделей обучается одной командой. Модель и граф выбираются в конфигурации, без правки кода.

Как проверить себя. Тесты 1 и 2 проходят. Отдельно обучите GCN ширины 16, с прореживанием 0,5, скоростью 0,01 и штрафом $5 \cdot 10^{-4}$. Опубликованная точность такой модели [6]: 81,5 % на Cora, 70,3 % на CiteSeer, 79,0 % на PubMed. Расхождение больше трёх процентных пунктов — повод искать ошибку. В экспериментах ширина равна 64. Эти проценты к ним не относят.

8.2 Задание 2. Статистики графа и мера сглаживания

Что сделать.

1. Функция `graph_stats` возвращает: N и $|E|$; среднюю, медианную и наибольшую степень; число компонент связности и изолированных вершин; размеры классов N_c; h_E, h_0 и $h_{\deg}$ (5.7), (5.8). Если рёбер нет, h_E не определена: функция возвращает `n/a`.
2. Статистики каждого графа, на котором модель обучалась или оценивалась, записывайте в паспорт запуска.
3. Функция `smoothing_report` для каждого слоя вычисляет S (5.15) по 20 000 пар вершин с фиксированным seed, $\hat{\mathcal{E}}$ (5.16) и точность зонда на `val`. Она работает и для распространения без обучения: $H = \hat{A}^l X$, $l = 0, \ldots, L$. Сводка сохраняется в `summary.json` запуска, по записи на слой.

Что должно получиться. Для любого графа строится таблица статистик, для любой модели — послойная сводка сглаживания.

Как проверить себя. После округления до сотых h_E совпадает с опубликованными значениями [9]: 0,81 для Cora, 0,74 для CiteSeer, 0,80 для PubMed. Без обучения $\hat{\mathcal{E}}(\hat{A}^l X)$ убывает с ростом l. Тест 5 проходит.

8.3 Задание 3. Возмущения графа и нулевые модели

Что сделать.

1. `delete_edges(edge_index, p, seed)` удаляет рёбра по (5.9). Готовая функция PyG `dropout_edge` подходит только с параметром `force_undirected=True`: по умолчанию она удаляет записи (i, j) и (j, i) независимо.
2. `add_edges(edge_index, p, seed)` добавляет $\lfloor p|E| \rfloor$ случайных пар вершин, которых нет в графе, без петель.
3. `degree_preserving_null(edge_index, Q, seed)` выполняет перестановки (5.11) и возвращает новый граф и долю успешных попыток.
4. `erdos_renyi(N, num_edges, seed)` строит граф из случайных пар вершин.
5. Функции не меняют признаки, метки и маски и возвращают статистики Задания 2. Параметры и seed возмущения входят в конфигурацию, поэтому любой изменённый граф восстанавливается по `EXP_ID`.

Что должно получиться. Любой возмущённый граф строится одним вызовом и воспроизводится по конфигурации.

Как проверить себя. Тесты 3 и 4 проходят. Гомофилия h_E нулевого графа близка к h_{deg}, а не к исходной h_E: для Cora — 0,17, а не 0,81. Доля успешных попыток на этих наборах выше 0,9.

Что добавляется в общий проект. Модули `src/graph/` (`datasets.py`, `stats.py`, `perturb.py`, `null_models.py`), `src/models/graph/` (`mlp.py`, `gcn.py`, `sage.py`, `gat.py`) и `src/diagnostics/oversmoothing.py`. Монитор градиентов из ЛР1 используется без изменений.

9 Эксперименты

Перед каждым экспериментом запишите гипотезу: что ожидаете увидеть и почему, со ссылкой на формулу раздела 5. E1 и E3 поисковые: достаточно одного seed. Главный вывод проверяет E2 на трёх seeds. Результат, разошедшийся с гипотезой, засчитывается, если объяснена причина. Режим везде трансдуктивный: укажите его в конфигурации и на графиках.

9.1 Эксперимент E1. Помогает ли граф

Вопрос. Даёт ли граф сведения сверх признаков вершин, и как качество зависит от доли сохранённых рёбер?

Что сделать.

1. Обучите перцептрон, GCN и вторую модель профиля. Вычислите G_{top} (5.12) для обеих графовых моделей.
2. Для GCN возьмите $p \in \{0;\ 0{,}1;\ 0{,}3;\ 0{,}5;\ 0{,}8;\ 1\}$ по (5.9) в двух режимах. *Оценка на повреждённом графе:* обучение на G, оценка на G_p без переобучения. *Обучение на повреждённом графе:* и обучение, и оценка на одном G_p.
3. Для каждого G_p запишите число изолированных вершин и h_E.
4. Постройте один график: $M(p)$ в двух режимах и горизонтальная линия M_{MLP}. При $p = 1$ GCN совпадает с перцептроном (тест 2), поэтому точка $p = 1$ второго режима должна лежать около этой линии.

Порог приёмки. При $p \in \{0{,}3;\ 0{,}5;\ 0{,}8\}$ число изолированных вершин отличается от предсказания (5.10) не более чем на 20 %. Если вершин заметно меньше, записи (i, j) и (j, i), вероятно, удаляются независимо. Величина G_{top} приведена для обеих моделей. Вывод «граф помогает» пока предварительный: его проверяет E2.

Граница применимости. При каком p кривая второго режима опускается до уровня перцептрона? Какая доля вершин `test` при этом изолирована? Убедитесь, что при $p < 1$ гомофилия h_E почти не меняется: граф перестаёт помогать из-за потери соседей, а не гомофилии.

9.2 Эксперимент E2. Нулевая модель с сохранением степеней вершин

Вопрос. Важна ли конкретная структура рёбер, или GCN хватит графа с теми же степенями вершин и случайными соседями?

Что сделать. До запусков запишите гипотезу в `hypothesis.md` и зафиксируйте файл в Git.

1. чем различаются конфигурации — только графом: G или G_{null};
2. что зафиксировано: признаки, маски, GCN глубины 2, гиперпараметры из E1, число эпох, seeds;
3. ожидаемые знак и величину $\bar{\Delta}$ со ссылкой на формулу;
4. какой результат опровергнет гипотезу.

Для каждого $s \in \{1, 2, 3\}$ постройте $G_{\text{null}}^{(s)}$ с $Q = 10$ и seed s. Запишите долю успешных перестановок, число компонент и h_E. С тем же seed обучите GCN на G, GCN на нулевом графе и перцептрон. Вычислите Δ_s (5.13). Постройте график точностей и трёх Δ_s.

Порог приёмки.

- Степени вершин в G и $G_{\text{null}}^{(s)}$ совпадают (тест 4).
- Гомофилия h_E каждого нулевого графа отличается от h_{deg} (5.8) не более чем на 20 %: рёбра больше не несут сведений о классах. Иначе ищите ошибку в перестановках.
- Вывод «структура рёбер важна» установлен, если все три Δ_s одного знака и $|\bar{\Delta}| \geq 2s_{\Delta}$. Иначе вывод формулируется как «различие не установлено». Приводятся все три Δ_s. Расхождение с гипотезой не снижает оценку, если оно объяснено.

Граница применимости. Нулевая модель сохраняет степени, поэтому она не проверяет, помогают ли сами степени. Повторите сравнение для seed 1 с графом Эрдёша–Реньи с тем же $|E|$: его h_E близка к h_0, а степени другие. Что это говорит о роли степеней? Три seeds описывают разброс только на данном разбиении.

9.3 Эксперимент E3. Глубина и чрезмерное сглаживание

Вопрос. С какой глубины GCN перестаёт выигрывать от новых слоёв, и чем вызвано ухудшение: сглаживанием, трудностью обучения или переобучением?

Что сделать.

1. Без обучения вычислите S и $\hat{\mathcal{E}}$ для $\hat{A}^l X$, $l = 0, \ldots, L_{\max}$, где $L_{\max}$ — наибольшая глубина профиля. Это предсказание (5.14) для сети без весов.
2. Для каждой глубины профиля обучите обычную GCN (без остаточных связей и нормализации) с seed 1 и гиперпараметрами E1. Запишите точности на `train` и `val`, а монитором ЛР1 — ρ_T по вершинам `train` и GI.
3. Для каждого слоя каждой модели получите сводку Задания 2.
4. Постройте графики: точности на `train` и `val` от L; $\hat{\mathcal{E}}$ по слоям для всех глубин в логарифмической шкале вместе с кривой без обучения; точность зонда по слоям.

Порог приёмки.

- Указана лучшая глубина $l^\star = \arg\max_L M_{\text{val}}(L)$, где M_{val} — точность на `val`. Падение M_{val} относительно $l^\star$ заметно, если оно больше $2s_m$. Здесь s_m — стандартное отклонение точности GCN на `val` по трём seeds E2 (правило ЛР0).
- Для каждой глубины с заметным падением названа одна причина. $\rho_T \geq 0{,}9$: сеть не обучилась (ЛР1); большой GI — затухание градиента, и о сглаживании судить нельзя. $\rho_T \leq 0{,}5$ и $\hat{\mathcal{E}}$ не ниже, чем при $l^\star$: переобучение. $\rho_T \leq 0{,}5$, а $\hat{\mathcal{E}}$ и точность зонда ниже, чем при $l^\star$: сглаживание. Иначе случай смешанный: укажите, какие признаки спорят.
- Если заметного падения до $L_{\max}$ нет, это явно указано.

Граница применимости. Где обученная сеть сглаживает сильнее кривой $\hat{A}^l X$, а где слабее? Повторите наибольшую глубину с остатком $H^{(l+1)} = H^{(l)} + \sigma(\hat{A}H^{(l)}W^{(l)})$. Почему формула (5.14) к ней прямо не применима?

10 Контрфактические вопросы

Ответ опирается на ваши числа со ссылками на `RUN_ID`. Если для ответа нужен дополнительный запуск, выполните его.

1. Сравните по числам E2 точность перцептрона и GCN на нулевом графе. Если GCN хуже, объясните это по формуле (5.4). Какой вес у представления самой вершины степени d_i? Обучите вторую модель профиля на $G^{(1)}_{\text{null}}$. Повторяется ли эффект и почему, по формуле (5.5) или (5.6)?
2. Предскажите h_E после добавления $\lfloor 0{,}5|E| \rfloor$ случайных рёбер по h_E и h_0. Проверьте и обучите GCN. Что опаснее: удалить половину рёбер (E1) или добавить столько же случайных?
3. На сколько упало качество в каждом режиме E1 при $p = 0{,}5$? Сколько вершин `test` стали изолированными? Малое падение значит, что модель не использует граф?
4. Найдите в E3 слой, где S уже высока, а точность зонда ещё близка к лучшей. Достаточно ли роста S? Что добавляют $\hat{\mathcal{E}}$ и зонд?
5. Определите $l^\star$ по точности на `test` вместо `val`. Совпала ли она с $l^\star$ по `val`? Почему так выбирать нельзя, хотя признаки и рёбра вершин `test` модель видит при обучении?

11 Обязательные тесты

1. Выход перцептрона не зависит от `edge_index`: на исходном, пустом и случайном графе он одинаков.
2. GCN на графе без рёбер и перцептрон с теми же весами дают одинаковый выход с точностью 10^{-6}: при $A = 0$ в (5.4) получается $\hat{A} = I$.
3. Для каждой записи (i, j) есть (j, i), петель и повторов нет. `delete_edges` при $p = 0$ сохраняет граф, при $p = 1$ удаляет рёбра. Возмущения не меняют признаки, метки и маски.

4. Перестановки (5.11) сохраняют степени всех вершин и не создают петель и повторов. Доля успешных попыток записывается и на графе из профиля не ниже 0,5.
5. На связном графе из 10 вершин строки $\hat{A}^{200}X$, делённые на $\sqrt{d_i+1}$, совпадают с точностью 10^{-4} (5.14). При $h_i = \sqrt{d_i+1}\,v$ мера $\hat{\mathcal{E}}$ меньше 10^{-8}; умножение H на число её не меняет.
6. Замена меток вершин `test` случайными не меняет обученные веса, выбранную эпоху и гиперпараметры.

12 Что сдать

Сдаются модули заданий 1–3, конфигурации E1–E3, файл `hypothesis.md`, тесты и отчёт. Отчёт состоит из шести частей:

1. **Профиль и протокол:** набор данных, вторая модель, глубины, гиперпараметры, режим; статистики исходного графа (Задание 2).
2. **E1:** G_{top} двух моделей; график $M(p)$; число изолированных вершин против (5.10); h_E повреждённых графов.
3. **E2:** гипотеза, статистики нулевых графов, все Δ_s, вывод, сравнение с графом Эрдёша–Реньи.
4. **E3:** таблица по глубинам, послойные графики, $l^\star$, причина падения.
5. **Карта зависимости качества от структуры графа:** таблица M для G, графа без рёбер, G_p при $p = 0{,}5$, нулевого графа, графа Эрдёша–Реньи и глубин $l^\star$, $L_{\max}$; зафиксированные факторы.
6. **Ответы на контрфактические вопросы.**

Каждое число сопровождается ссылкой на `RUN_ID`. Графики строятся из сохранённых результатов.

13 Критерии завершения и оценивания

Работа завершена, если одновременно:

- эксперименты E1–E3 выполнены, у каждого результата есть `RUN_ID`;
- тесты проходят;
- файл `hypothesis.md` зафиксирован в Git до запусков E2;
- статистики графов и параметры возмущений записаны в паспорта запусков;
- карта зависимости качества от структуры графа построена;
- на все контрфактические вопросы даны ответы с собственными числами.

Веса частей оценки те же, что в ЛР0: реализация 0,15, экспериментальный протокол 0,30, диагностика 0,30, перенос 0,25.

14 Дополнительные задания

1. **Гомофилия и гетерофилия.** Блочная модель [17] (stochastic block model): 4 равных класса, 2000 вершин, средняя степень 10, поэтому $p_{\text{in}} + 3p_{\text{out}} = 0{,}02$ и $\mathbb{E}[h_E] \approx p_{\text{in}}/(p_{\text{in}} + 3p_{\text{out}})$. Возьмите уровни $h_E \in \{0{,}9;\ 0{,}6;\ 0{,}3;\ 0{,}1\}$. Признаки — центр класса плюс шум того же масштаба. Сравните $M(h_E)$ перцептрона, GCN и второй модели [18].
2. **Граф по ключевым точкам лица.** В пакете эмоций курса 68 точек схемы 300-W [19], 63 анатомических ребра, 8 кадров на клип и разбиение по актёрам («Протокол данных курса»). Признаки точки — координаты минус центр лица, делённые на расстояние между центрами глаз. Сравните по macro-F1 на клипах анатомический граф, граф четырёх ближайших соседей, полный граф, нулевую модель и перцептрон. Выигрывает ли анатомический граф у нулевого [20]?
3. **Аудит внимания GAT.** Сравните средние α_{ij} на рёбрах внутри классов и между ними; оба конца ребра — в `train`. Без переобучения удалите 10 % рёбер с наибольшим α_{ij} и столько же случайных рёбер, по три seeds. Если первое снижает M не сильнее, вес внимания не доказывает важность ребра [21].

15 Контрольные вопросы

1. Что такое передача сообщений и от каких вершин зависит представление вершины после L слоёв?
2. Чем агрегация GCN отличается от GraphSAGE при чужих классах соседей?
3. Зачем в GCN петли и что происходит с изолированной вершиной?
4. Что такое гомофилия и зачем сравнивать h_E с уровнем случайности?
5. Что такое нулевая модель графа и почему в ней сохраняют степени?
6. К чему сходится $\hat{A}^L X$ и почему это называют чрезмерным сглаживанием?
7. Чем сглаживание отличается от переобучения и от затухания градиента?
8. Чем трансдуктивный режим отличается от индуктивного для вершин `test`?

16 Итог

Итог работы — не рейтинг архитектур, а ответ, что именно даёт граф. Прирост над перцептроном — необходимое условие. Сравнение с нулевой моделью показывает, важна ли конкретная структура рёбер. У глубины есть предел: дальше передача сообщений стирает различия вершин. Качество сети само по себе не доказывает пользу графа.

Литература

[1] Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. Погружение в мир нейронных сетей. — СПб.: Питер, 2018. — 480 с.

[2] Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение / пер. с англ. А. А. Слинкина. — 2-е изд., испр. — М.: ДМК Пресс, 2018. — 652 с.

[3] PyTorch Geometric Documentation: Introduction by Example. https://pytorch-geometric.readthedocs.io/en/latest/get_started/introduction.html

[4] C. Fourrier. Introduction to Graph Machine Learning. Hugging Face Blog, 2023. https://huggingface.co/blog/intro-graphml
[5] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, G. E. Dahl. Neural Message Passing for Quantum Chemistry. *ICML*, 2017.
[6] T. N. Kipf, M. Welling. Semi-Supervised Classification with Graph Convolutional Networks. *ICLR*, 2017.
[7] W. L. Hamilton, R. Ying, J. Leskovec. Inductive Representation Learning on Large Graphs. *NeurIPS*, 2017.
[8] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio. Graph Attention Networks. *ICLR*, 2018.
[9] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, D. Koutra. Beyond Homophily in Graph Neural Networks: Current Limitations and Effective Designs. *NeurIPS*, 2020.
[10] S. Maslov, K. Sneppen. Specificity and Stability in Topology of Protein Networks. *Science*, 296(5569):910–913, 2002.
[11] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann. Pitfalls of Graph Neural Network Evaluation. arXiv:1811.05868, 2018.
[12] Q. Li, Z. Han, X.-M. Wu. Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning. *AAAI*, 2018.
[13] K. Oono, T. Suzuki. Graph Neural Networks Exponentially Lose Expressive Power for Node Classification. *ICLR*, 2020.
[14] C. Cai, Y. Wang. A Note on Over-Smoothing for Graph Neural Networks. arXiv:2006.13318, 2020.
[15] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, T. Eliassi-Rad. Collective Classification in Network Data. *AI Magazine*, 29(3):93–106, 2008.
[16] Z. Yang, W. W. Cohen, R. Salakhutdinov. Revisiting Semi-Supervised Learning with Graph Embeddings. *ICML*, 2016.
[17] P. W. Holland, K. B. Laskey, S. Leinhardt. Stochastic Blockmodels: First Steps. *Social Networks*, 5(2):109–137, 1983.
[18] Y. Ma, X. Liu, N. Shah, J. Tang. Is Homophily a Necessity for Graph Neural Networks? *ICLR*, 2022.
[19] C. Sagonas, G. Tzimiropoulos, S. Zafeiriou, M. Pantic. 300 Faces in-the-Wild Challenge: The First Facial Landmark Localization Challenge. *ICCV Workshops*, 2013.
[20] Q. T. Ngoc, S. Lee, B. C. Song. Facial Landmark-Based Emotion Recognition via Directed Graph Neural Network. *Electronics*, 9(5):764, 2020.
[21] B. Knyazev, G. W. Taylor, M. Amer. Understanding Attention and Generalization in Graph Neural Networks. *NeurIPS*, 2019.